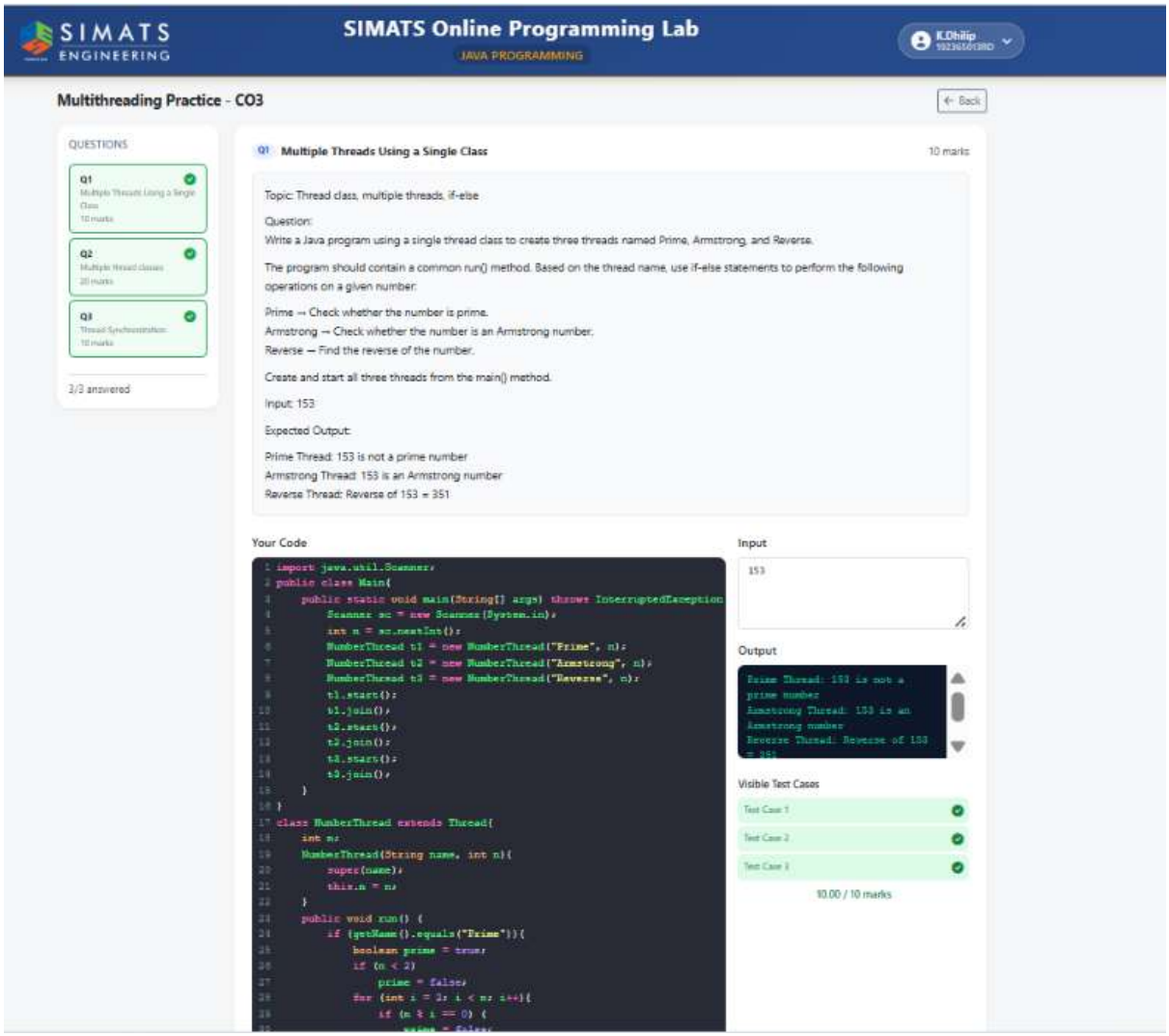
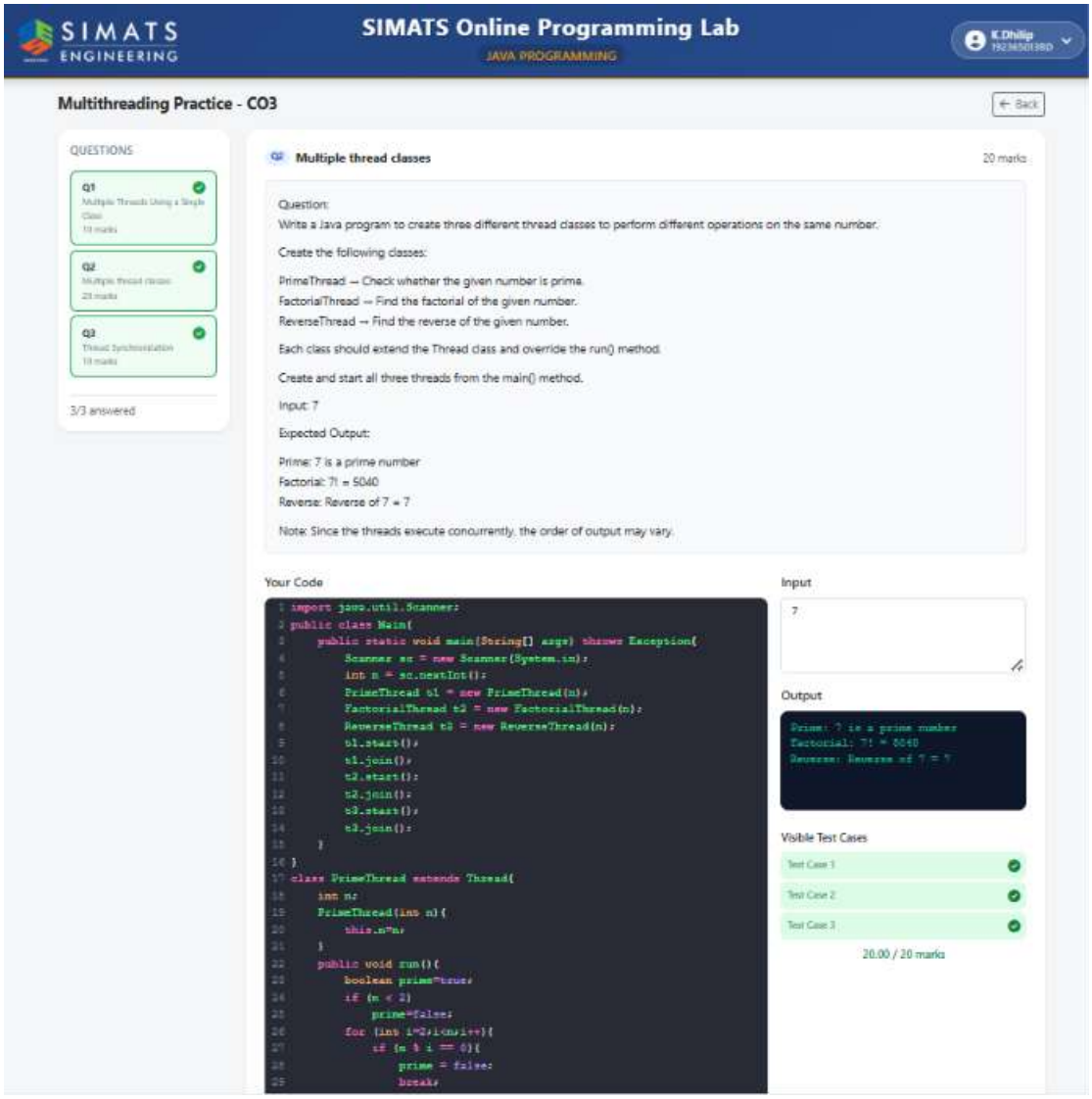
Multithreading Practice – CO3

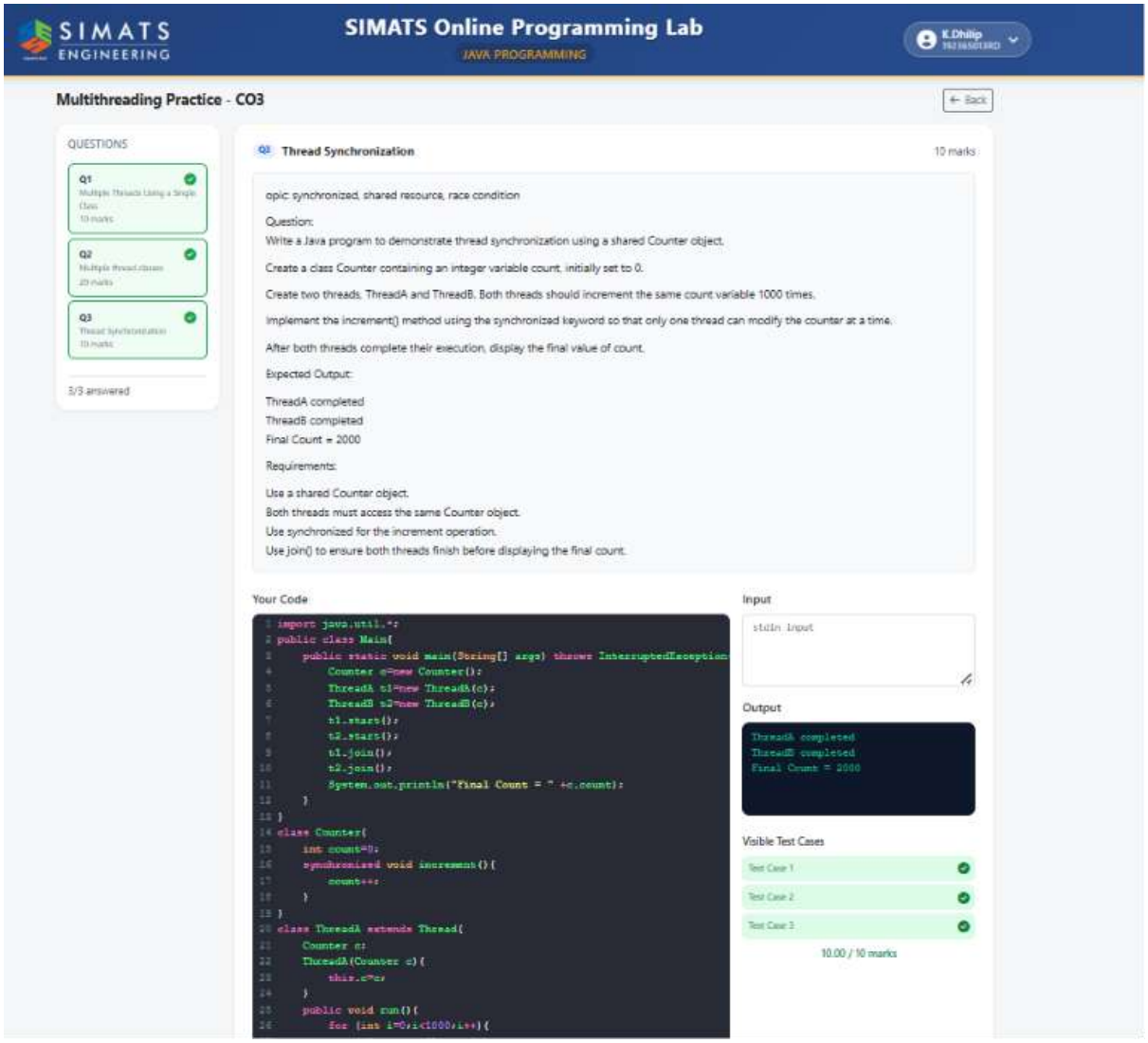
1.



2.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
K.Dhilip
Multithreading Practice - CO3
← Back
QUESTIONS
Q1
Q2
Q3
3/3 answered
Multiple thread classes
20 marks
Question:
Write a Java program to create three different thread classes to perform different operations on the same number.
Create the following classes:
PrimeThread – Check whether the given number is prime.
FactorialThread – Find the factorial of the given number.
ReverseThread – Find the reverse of the given number.
Each class should extend the Thread class and override the run() method.
Create and start all three threads from the main() method.
Input: 7
Expected Output:
Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7
Note: Since the threads execute concurrently, the order of output may vary.
Your Code
import java.util.Scanner;
public class Main{
public static void main(String[] args) throws Exception{
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
PrimeThread t1 = new PrimeThread(n);
FactorialThread t2 = new FactorialThread(n);
ReverseThread t3 = new ReverseThread(n);
t1.start();
t1.join();
t2.start();
t2.join();
t3.start();
t3.join();
}
}
class PrimeThread extends Thread{
int n;
PrimeThread(int n){
this.n=n;
}
public void run(){
boolean prime=true;
if (n < 2)
prime=false;
for (int i=2;i<n;i++){
if (n % i == 0){
prime = false;
break;
Input
7
Output
Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7
Visible Test Cases
Test Case 1
Test Case 2
Test Case 3
20.00 / 20 marks

3.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
E.Dhilip
Multithreading Practice - CO3
← Back
QUESTIONS
Q1
Q2
Q3
3/3 answered
Q3 Thread Synchronization
10 marks
opic synchronized, shared resource, race condition
Question:
Write a Java program to demonstrate thread synchronization using a shared Counter object.
Create a class Counter containing an integer variable count, initially set to 0.
Create two threads, ThreadA and ThreadB. Both threads should increment the same count variable 1000 times.
Implement the increment() method using the synchronized keyword so that only one thread can modify the counter at a time.
After both threads complete their execution, display the final value of count.
Expected Output:
ThreadA completed
ThreadB completed
Final Count = 2000
Requirements:
Use a shared Counter object.
Both threads must access the same Counter object.
Use synchronized for the increment operation.
Use join() to ensure both threads finish before displaying the final count.
Your Code
1 import java.util.*;
2 public class Main{
3 public static void main(String[] args) throws InterruptedException
4 Counter c=new Counter();
5 ThreadA t1=new ThreadA(c);
6 ThreadB t2=new ThreadB(c);
7 t1.start();
8 t2.start();
9 t1.join();
10 t2.join();
11 System.out.println("Final Count = " +c.count);
12 }
13 }
14 class Counter{
15 int count=0;
16 synchronized void increment(){
17 count++;
18 }
19 }
20 class ThreadA extends Thread{
21 Counter c;
22 ThreadA(Counter c){
23 this.c=c;
24 }
25 public void run(){
26 for (int i=0;i<1000;i++){
Input
stdin input
Output
ThreadA completed
ThreadB completed
Final Count = 2000
Visible Test Cases
Test Case 1
Test Case 2
Test Case 3
10.00 / 10 marks